Birla Institute of Technology & Science, Pilani
Second Semester 2015-2016, CS F111 Computer Programming
Comprehensive Exam (Open Book) *PART A Set X*

Suggested time: 60 minutes **08/5/16, 2.00 - 4.00 PM** **Max. Marks: 44**

NOTE: 1. There are 22 questions in Part A. Each question has exactly one answer correct and carry 2M.
2. There is no negative marking. Overwritten answers will not be rechecked.
3. For each question, write the correct option (A, B, C, or D) at the designated place on the back of this sheet.
4. In each question, choose the best option. Assume required header files are included.

ID **NAME**

Q1. What is the output of the following?

```
void pass (int a) {
  a = 20;
}
void main() {
  int *n;
  *n = 10;
  pass (*n);
  printf ("%d",*n);
}
```

(A) 10 (B) 20
(C) 2010
(D) The code has an Error.

Q2. What is the output of the following?

```
union myunion{
  int age;
  char initial;
};
void main() {
  union myunion u;
  u.age = 8;
  u.initial = 'R';
  printf("%d", u.age);
}
```

(A) 114 (B) 82 (C) 81 (D)Cannot print as integer

Q3. What is the output of the following program

```
void main () {
  int a[5] = { 5, 1, 15, 20, 25 };
  int i, j, k = 1, m;
  i = ++a[1];
  j = a[1]++;
  m = a[i++];
  printf ("%d %d %d\n", i, j, m);
}
```

(A) 2 2 1
(B) 3 2 15
(C) 6 7 20
(D) 6 7 15

Q4. Choose the best option w.r.t. the following program:

```
void main() {
  enum day {A=6, G=7,M,C,I=11,F=3,K};
  enum day d1 = A;
  printf ("%d",strcmp (d1,"A")); //Line 1
  int b = A + F * I / C % M;
  printf ("%d",b); //Line 2
}
```

(A) Line 1 prints 1 and Line 2 prints 9
(B) Line 1 has an error and Line 2 prints 9
(C) Line 1 prints 1 and Line 2 has an error.
(D) Line 1 and Line 2 both has an error.

Q5. What is the output of the following?

```
void main() {
  int ary[4] = {1, 2, 3, 4};
  int *p = ary + 3;
  printf("%d\n", p[-2]);
}
```

(A) 1
(B) 2
(C) compilation error
(D) garbage value

Q6. Given:

```
int arr[4][4];
int *ptr = arr;
```

What value should be added to ptr to reach the array location arr[2][2]. Assume that integer require 4 bytes.

(A) 10 (B) 40
(C) 11 (D) 44

Q7. The output of which statement (SA and/or SB) is 50?

```
void main() {
int a[]={10, 20, 30, 40, 50};
 char *p;
 p= (char*) a;
 printf("%d",*((int*)p+4)); //SA
printf("%d",*(p+sizeof(int)*4)); //SB
}
```

(A) SA only (B) Both SA and SB
(C) SB only (D) Neither SA, nor SB

Q8. What is the output?

```
int main(){
  int i;
  for (i = 0; i < 10; ++i)
    if ((i % 2) == 0) printf("A");
    else if ((i % 3) == 0) printf("B");
    else printf("C");
}
```

(A) AABCACABAC (B) ABCABCABCB
(C) ACBBCABCAB (D) ACABACACAB

Q9. What is the output?

```
struct rectangle {
  int len;
  int wid;
};
void call_me (struct
rectangle rect) {
  rect.len = 30;
  rect.wid = 40;
}
int main() {
struct rectangle *ptr, r;
ptr = &r;
r.len = 10;
r.wid = 20;
call_me (*ptr);
printf ("%d,%d",
(*ptr).len, (*ptr).wid);
}
```

(A) 10 , 20 (B) 20 , 10
(C) 30 , 40 (D) 40 , 30

Q10 Consider the following declarations:

```
struct test {
    int num;
    float j;
  };
  struct test arr[5];
```

Which of the following four statements (S1 to S4) is correct to access element num in arr[2]?

S1: arr[2].num
S2: (arr+2).num
S3: (arr+2) -> num
S4: (*(arr + 6/3)).num

(A) S1, S2
(B) S1, S3
(C) S1, S4
(D) S1, S3, S4

Q11 What is the output?

```
int main() {
  int i = 0, j = 0, k=0;
  for (;k<1;k++)
   for (i; i < 2; i++){
    for (j = 0; j < 3; j++){
      printf("1");
      break;
    }
    printf("2");
   }
  printf("3");
}
```

(A) 123 (B) 12123
(C) 123123 (D) 1

Q12 What is the output?

```
void main() {
  char ch[] = "Jaipur";
  ch[0]='K';
  printf("%s, ", ch);
  strcpy(ch,"Kanpur");
  printf("%s", ch+1);
}
```

(A) Kaipur, Kanpur
(B) Jaipur, Kanpur
(C) Kaipur, anpur
(D) There is a syntax error.

Q13 What is the output if it is executed from the command line as follows? Assume myOutput is the output file.

cmd> myOutput Mon Tues Wed

```
void main(int argc, char *argv[]) {
  while(argc)
    printf("%s", argv[--argc]);
}
```

(A) myOutput Mon Tues Wed
(B) myOutput Mon Tues
(C) Wed Tues Mon myOutput
(D) Wed Tues Mon

Q14. What is the output when recurse is called from main() ?

```
void recurse() {
  static int n = 987654321;
  if (n == 0)
     return;
  printf ("%d",n%10);
  n /= 100;
  int a = n;
  recurse();
  if (a!=0)
    printf ("%d",a%10);
}
```

(A) 135799753
(B) 1357913579
(C) 1357933333
(D) It loops forever

Q15. What is the output, if user enters strings "one", "two", and "three" ?

```
int main() {
  char *a[4]; int i = 0;
  while(i < 3) {
    char b[50];
    scanf(" %s", b);
    a[i++] = b;
  }
  i = 0;
  while(i < 3)
    printf(" %s ", a[i++]);
}
```

(A) one two three
(B) ott
(C) three three three
(D) one one one

Q16. The desired output of the following program is 1cpe. Which is correct if-condition for it?

```
void main() {
 char c[4][4] =
{{'c','s','f','1'},
{'1','1','c','o'},
{'m','p','r','e'},
{'e','x', 'a', 'm'}};
 int i, j;
 for(i = 0; i < 4; i++)
  for(j = 4-i; j >= 0; j--)
   if([redacted])
    printf("%c",c[i][j]);
}
```

(A) i == j (B) (i + j) == 3
(C) i != j (D) (i + j) == 4

Q17. What is the output?

```
void main() {
 int i=0;
 if(i==0) {
  i=((5,(i=3)),i=1);
  printf("%d",i);
 }
 else
  printf("equal");
}
```

(A) 1
(B) 3
(C) 5
(D) equal

Q18. Consider a non-empty singly linked list with odd number of nodes. Node of the list contains integer (in variable data) and address of next node (in variable next). What does the following function do if head is the address of the first node of the list?

```
void fun1(struct node* head) {
 if(head->next == NULL) {
   printf("%d ", head->data);
   return;
 }
 fun1(head->next->next);
 printf("%d ", head->data);
}
```

(A)Prints data field of all nodes in forward direction.
(B)Prints data field of all nodes in reverse order
(C)Prints data field of alternate nodes in forward direction.
(D)Prints data field of alternate nodes in reverse order

Q19. Assuming file small.text contains Bits Pilani, choose the correct option.

```
int main(){
char ch;
FILE *fp1,*fp2;
fp1=fopen("small.text","r");
fp2=fopen("big.text","w");
while((ch=fgetc(fp1)) !=EOF)
fputc(toupper(ch),fp2);
return 0;
}
```

(A) BITS PILANI will be copied to file big.text file.
(B) Runtime Error
(C) Compiler Error
(D) bITS pILANI will be copied to file big.text file.

Q20. What is the output?

```
void main() {   /* Assume 2 byte integer*/
  unsigned int i = 65536;
  while(i != 0)
    printf("%d,",++i);
}
```

(A) Infinite loop (B) 0 1 2 ... 65535
(C) 0, 1, 2, ... 32767, 32766, 32765, 1, 0
(D) No output

Q21. Which statement match the below code?

```
void main() {
 int input;   char output;
 scanf("%d", &input);
 output = ((input % 3) == 0) + '0';
 printf("%c", output); }
```

(A) Prints 1 if the input number is divisible by 3 Otherwise prints nothing.
(B) Prints 1 if the input number is divisible by 3 Otherwise prints 0
(C) Displays the ASCII value of 0
(D) None of the above

Q22. Which of the statements (S1 - S3) are correct about the below declarations?

char *q = "James";   char p[] = "James";

S1. There is no difference in the declarations and both serve the same purpose.
S2. The base address of q can be changed, whereas of p cannot be changed.
S3. In both cases the '\0' will be added at the end of the string "James".

(A) S1 and S2 (B) S1 and S3 (C) S3 only (D) S2 and S3

PART A Set X
ANSWER AREA (OPEN BOOK)
Write your answers legibly in the space provided below.

Q. No.	1	2	3	4	5	6	7	8	9	10	11
Correct Option											

Q. No.	12	13	14	15	16	17	18	19	20	21	22
Correct Option											

Birla Institute of Technology & Science, Pilani
Second Semester 2015-2016, CS F111 Computer Programming
Comprehensive Exam (Open Book) ***PART B***

Date: May 08, 2016 **Time: 2.00 - 4.00 PM** **Max. Marks: 46**

NOTE: Each question is partially implemented. ***Write the missing parts as your answers at the designated place in the separate answer sheet provided. Write only one statement in each box. Assume suitable header files are included.***

Q1. FIGURE 1 shows a program to read a number **n** from user and store all possible permutations of 1, 2, ..., n in a file. Overall there are n! permutations possible. For example, if n is 3, there will be 6 permutations. 1 2 3, 1 3 2, 2 1 3, 2 3 1, 3 2 1, 3 1 2.

We want to store all permutations in a file "Permutation.txt". Each permutation is stored in a newline in the file and there is a space between any two numbers. First we are storing the numbers 1 to n in an array **v** and use a function "**perm**" to store recursively all possible permutations. This function takes the array **v, i, n**, and file pointer **fp**, and generates the permutations of the array from element i to element n-1; and if i is equal to n then the function stores the array in the file. [**14 Marks**]

```
void swap (int v[], int i, int j) {
    int t;
    t = v[i];
    v[i] = v[j];
    v[j] = t;
}
void perm (int v[], int i, int n, FILE *fp)
{
    int j;
    if [Q1.a  2M] {
     for (j=0; j<n; j++) {
        [Q1.b  2M]
        putc(' ',fp);
     }
     [Q1.c  2M]
    }
    else {
        for (j=i; j<n; j++) {
          swap (v, i, j);
          [Q1.d  4M]
          swap (v, i, j);   } //End for
        }
}
main(){
 int n;
 FILE *fp;
 scanf("%d", &n);
 int v[n],i;
 fp=[ Q1.e  2M]
 for(i=0;i<n;i++)
     v[i]=i+1;
 [Q1.f  2M]
 fclose(fp);
}
```

FIGURE 1

Q2. A character string is said to have period k if it can be formed by concatenating one or more repetitions of another substring of length k. For example, the string "abcabcabcabc" has period 3, since it is formed by 4 repetitions of substring "abc". It also has periods of 6 (two repetitions of "abcabc") and 12 (one repetition of "abcabcabcabc").

Complete the program shown in FIGURE 2 which reads a character string and determine its smallest period. [**14 Marks**]

```
int finder(char *str)
{
   int len = strlen(str), period, flag, i, j;
   char q[100];
   for (period=1 ; period<=(len/2) ; period++)
   {
      for (i=0 ; i<period ; i++) //Copy period
                              number of characters in q
       [Q2(a)  2M]
      flag=1;
      for (i=0 ; i<len  && flag ; [Q2(b)  2M] )
      {
         for (j=0 ;[Q2(c)          2M] ; j++)
         {
            if ( [Q2(d)          2M] )
             flag = 0 ;
         }
      }
      if (flag) return[Q2(e)  2M]    ;
   }
   return[Q2(f)  2M];
}
int main() {
   char input[200];
   gets(input);
   printf("%d\n", [Q2(g)          2M]    );
   return 0;
}
```

FIGURE 2

Q3. Consider two classes of chemicals: **OH** and **NON_OH**. **OH** class of chemicals are those whose chemical names end with "OH" and **NON_OH** class are those whose chemical names does not end with "OH". Consider a singly Linked List (corresponding structures shown in *FIGURE-3*) in which a node stores the chemical name (**chem_name**) and the atomic weight (**weight**) of the chemical. The nodes in the list are maintained such that all **OH** class chemicals come before than all **NON_OH** class chemicals. Further, among all **OH** class chemicals, the nodes are sorted in increasing order of atomic weight. Similarly, among all **NON_OH** class chemicals, the nodes are sorted in increasing order of atomic weight. The Head node of the linked list stores a total number of **OH** class chemicals (in variable **OH_count**) and a total number of **NON_OH** class chemicals (in variable **non_OH_count**) in the linked list. With respect to this, complete the following two functions. In each box, write only one C statement.

(a) Function **string_with_OH()**, shown in FIGURE-4,which takes a chemical name as input argument and returns 1 if the chemical name is in OH class; and 0 (zero) if it is in NON_OH class.

(b) Function **Add_Node ()** , shown in FIGURE-5, which takes the Head node of linked list, chemical name, and its atomic weight as input arguments. This function adds a new node (with input arguments chemical name and atomic weight) into the list as per the sorting rules specified above. Initially, there will be no chemical in the list. This function can be called repeatedly to insert a chemical in to the list such that the chemical gets inserted as per the order specified above.

[**Note**: Assume that there is only one node in the list corresponding to a chemical name, each chemical name has a unique atomic weight, and chemical name will always be written using upper case characters.] [**18 Marks**]

```
struct node;
typedef struct node NODE;
typedef NODE* PTR;

struct node{
char *chem_name;
int weight;
PTR link;
};

typedef struct {
int OH_count;
int non_OH_count;
PTR head;
}HNODE;
```

FIGURE-3

```
int string_with_OH(char *str) {
int len;
len=strlen(str);
if( [Q3(a)   1M] )
   return 1;
else
    return 0;
}
```

FIGURE-4

```
HNODE Add_Node(HNODE h, char *name, int wt) {
 PTR current, newnode, previous;
 int cnt;
 newnode = (PTR)malloc(sizeof(NODE));
 newnode->chem_name = [Q3(b)   2M];
 strcpy(newnode->chem_name , [Q3 (c)]   );
 newnode->weight=wt;
 newnode->link = NULL;
 if(h.head==NULL ){
   h.head=newnode;
   if(string_with_OH(name))
      h.OH_count = 1;
   else
     h.non_OH_count = 1;
  return h;
 } //End if(h.head==NULL )
 else if(string_with_OH(name))  {
   cnt = 1;
   current = previous = h.head;
   while(cnt <= h.OH_count &&[Q3(d)   1M] ) {
     previous = current;
     current = current->link;
     cnt++;
   } //End while
  if(previous == current)   {
     [Q3(e)   1M]
     [Q3(f)   1M]
   }
  else {
     [Q3(g)   1M]
     [Q3(h)   1M]
   }
   h.OH_count++;
   return h;
  }// END else if(string_with_OH(name))
  else if(string_with_OH(name)==0) {
     current = previous = h.head;
     cnt=1;
     while(cnt <= h.OH_count) {
       [Q3(i)   1M]
       [Q3(j)   1M]
       [Q3(k)   1M]
   } //END while
   while([Q3(l)   2M] ) {
          previous = current;
          current = current->link;
   } //END while
   if(previous == current) {
        [Q3(m)   1M]
        [Q3(n)   1M]
      }
     else {
       [Q3(o)   1M]
       [Q3(p)   1M]
      }
  h.non_OH_count++;
 } //END if(string_with_OH(name)==0)
 return h;
}
```

FIGURE-5